



Синтез и анализ на алгоритми за факултети ФКСТ, ФТК, ФЕТТ

проф. д-р Даниела Минковска
каб. 2324, кат. ПКТ, ФКСТ, ТУ-София
daniela@tu-sofia.bg



Лекция 8

Алгоритми за сортиране. Сортиране чрез избор, сливане, разпределение и паралелно сортиране. Външно сортиране.

“АЛГОРИТМИ + СТРУКТУРИ ОТ ДАННИ = ПРОГРАМИ”

Н. УИРТ



Сортиране. Основни групи

Най-общо могат да се обобщят следните **групи методи за сортиране**:

- Сортиране чрез **вмъкване** (*Insertion sort*);
- Сортиране чрез **размяна** (*Shaker sort*);
- Сортиране чрез **избор** (*Selection sort*);
- Сортиране чрез **сливане** (*Merge Sort*);
- Сортиране чрез **разпределение**.

Ще разгледаме няколко алгоритъма от изброените групи методи и тяхната реализация.



Сортиране. Сортиране чрез избор.

Сортиране чрез прост избор (извличане, пряка селекция):

Към тази група се отнасят: сортиране чрез прост избор, избор от дърво, пирамидална сортировка и др..

1. Сортиране чрез прост избор (пряка селекция)

Идея на метода: **търси** се **елемент с най-малкия** (или най-големия) **ключ**; **намереният елемент** се **премества в началото** (или в края) на масива **и се изключва от разглеждането**; действието се **повтаря, докато не се изберат всички n елемента**. Методът е аналогичен на метода на простото вмъкване - и в двата случая в левия край на масива се образува поредица от вече подредените елементи.

Сложността на метода е $O(n^2)$.





Сортиране. Сортиране чрез избор.

Сортиране чрез избор (извличане) – *Selection sort*:

Както и при метода на прекия избор, масива се разделя на **сортирана** и **несортирана** част, като на всяка стъпка на алгоритъма **сортираната област се разширява **отдясно**** с един **елемент**.

На първата стъпка се намира минималния елемент на масива и се разменя с първия. На втората, минималния измежду останалите елементи се разменя с втория елемент на масива и т. н. **На всяка следваща стъпка минималния елемент от несортираната част на масива си разменя мястото с първия елемент от несортираната част**, разширявайки по този начин сортираната част, докато тя не обхване целия масив.





Сортиране. Сортиране чрез избор.

Сортиране чрез прост избор (извличане, пряка селекция):

```
void straight_selection (int a[], unsigned n)
{
    unsigned i, j;
    int rab;
    for (i=0; i<n-1; i++)
        for (j=i+1; j<=n; j++)
            if (a[i] > a[j])
            {
                rab=a[i];
                a[i]=a[j];
                a[j]=rab;
            }
}
```





Сортиране. Сортиране чрез избор.

Сортиране чрез прост избор (извличане, пряка селекция):

Широко разпространен е и модифициран вариант на разгледаната реализация: във вътрешния цикъл е спестено адресирането на j -ия елемент като $a[j]$ и за целта е въведена променливата x :

```
void straight_selection (int a[], unsigned n)
{ unsigned i, j, ind;
  int x;
  for (i=0; i<n-1; i++)
    for (x=a[ind=i], j=i+1; j<=n; j++)
      if (a[j] < x)
        { x=a[ind=j];
          a[ind]=x;
          a[j]=x;
        }
}
```





Сортиране. Сортиране чрез избор.

Сортиране чрез прост избор (извличане, пряка селекция):

Подобно на сортирането по метода на мехурчето, алгоритъмът на пряката селекция винаги извършва **$n*(n-1)/2$ сравнения**. Но той е за предпочитане пред метода на мехурчето по броя на извършваните размени:

Straight Selection	Минимален	Среден	Максимален
С (бр. сравнения)	$n(n-1)/2$	$n(n-1)/2$	$n(n-1)/2$
М (бр. размени)	$3(n-1)$	$n \ln n$	$[n^2/4] + 3(n-1)$

Сортирането чрез пряк избор се нарежда сред **най-неефективните**, подобно на метода на мехурчето и метода на прякото вмъкване. Основен негов **недостатък е, че на i -та стъпка винаги се извършват точно $n-i$ сравнения, независимо от входните данни.**



Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

2. *Пирамидално сортиране (Heapsort)*

Методът се базира на **последователното избиране на най-малкия ключ измежду n елемента**, за което са необходими **$n-1$ сравнения**. След това на втората стъпка на алгоритъма **най-малкия ключ измежду $n-1$ елемента се избира с $n-2$ сравнения** и т.н.

Извод: на i -та стъпка ще трябва **$n-i$ сравнения**. Но не точно, т.к в резултат на прилагането на първата стъпка получаваме не само най-малкия елемент, но и редица съотношения между двойки от останалите елементи, които можем да съхраним и да използваме на втората стъпка. Следователно, колкото повече допълнителни съотношения се получат, толкова по-малко сравнения ще бъдат необходими на следващите стъпки, т. е. **колкото е по-ниско и по-разклонено дървото на сравненията, толкова повече информация ще носи то.**



Сортиране. Сортиране чрез избор.

Сортиране чрез избор от дърво:

За да се поясни последния извод, нека да разгледаме ново понятие - **двоично дърво на сравненията**.

Пример: сравнение на три елемента - a , b и c . Това е прост метод за **сортиране чрез сравнение** (основава се на поредица от сравнения, всяко от които ни носи допълнителна информация). Процесът продължава до пълното сортиране на множеството $\{a, b, c\}$. В зависимост от резултата от сравнението се получават различни съотношения между елементите, и различно продължение на процеса. Всяко **сравнение има два изхода - “да” и “не”, следователно** дава възможност процесът на сравнение да се изобрази като двоично дърво, в чиито листа се намират сортираните последователности, а в останалите възли - сравнения между двойки елементи от множеството:

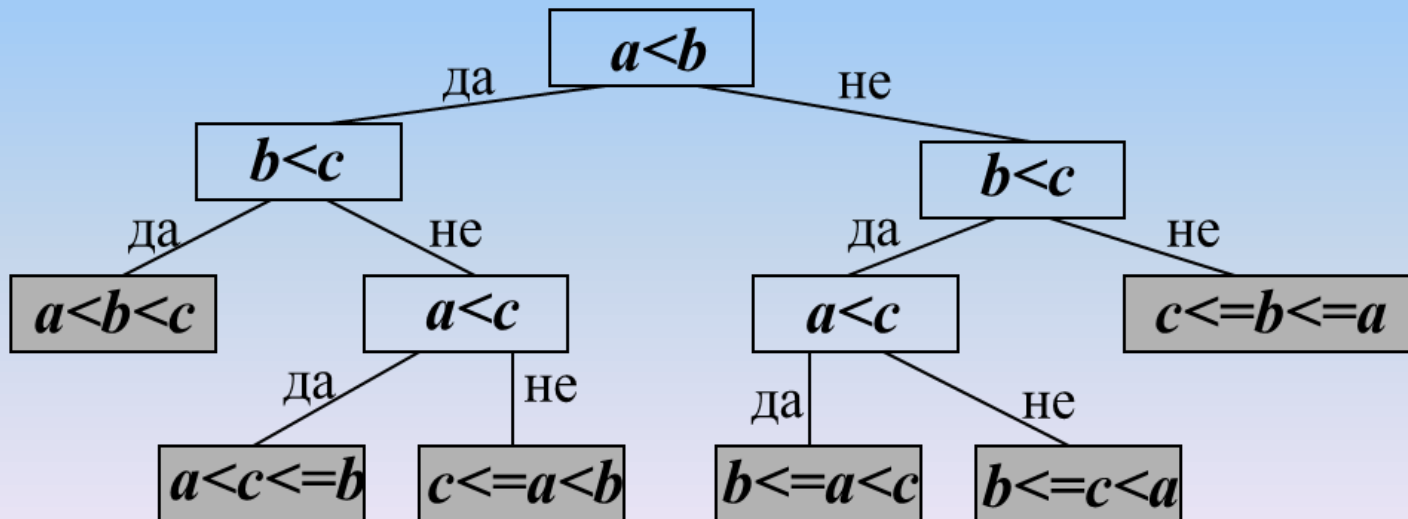




Сортиране. Сортиране чрез избор.

Сортиране чрез избор от дърво:

Всяко **сравнение** има два изхода - “**да**” и “**не**”, което ни дава възможност **процесът на сравнение** да се изобрази като **двоично дърво**, в чиито **листа** се намират **сортираните последователности**, а **в останалите възли** - **сравнения между двойки елементи от множеството**:





Сортиране. Сортиране чрез избор.

Сортиране чрез избор от дърво:

Дървото ни дава следния алгоритъм за сортиране на множеството:

```
if (a<b)
    if (b<c) cout<<a<<b<<c;
    else
        if (a<c) cout<<a<<c<<b;
        else cout<<c<<a<<b;
else
    if (b<c)
        if (a<c) cout<<b<<a<<c;
        else cout<<b<<c<<a;
    else cout<<c<<b<<a;
```





Сортиране. Сортиране чрез избор.

Сортиране чрез избор от дърво:

Алгоритъмът е много **нагледен**, но е почти **неприложим**, т.к броят на възможните изходи (листата на дървото) е **$n!$** (колкото възможните пермутации на изходното множество).

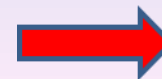
Затова се налага да се търсят други по-ефективни методи за сортиране. Но идеята на дървото може да послужи като **математически модел** за изграждане на други алгоритми.



Сортиране. Сортиране чрез избор.

Сортиране чрез избор от дърво:

Може да изгради дърво на сравненията, използвайки механизма на **турнира с елиминирание**, построяващ дървото на сравненията от листата към корена. В първия кръг на турнира се играят срещите: $x_1:x_2, x_3:x_4, \dots, x_{n-1}:x_n$. Във втория кръг играят **двойки победители от предишния кръг** и т. н. На **финала** се срещат двама участници и определят шампиона на турнира. Ако **в i -ия кръг** на турнира **броя на участниците е нечетен** (а това ще се случи поне веднъж, ако n не е степен на 2), **то един от играчите почива**, като автоматично се класира за следващия кръг. Използвайки посочената **стратегия за определяне на минималния елемент** на масива се получава следното: в **първия кръг** на турнира **с $n/2$ сравнения** се **определя минималния елемент** за всяка двойка, с **още $n/4$ сравнения** се **определя минималния елемент** между две **двойки** (т. е. 4 елемента) и т. н.



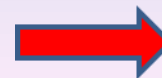
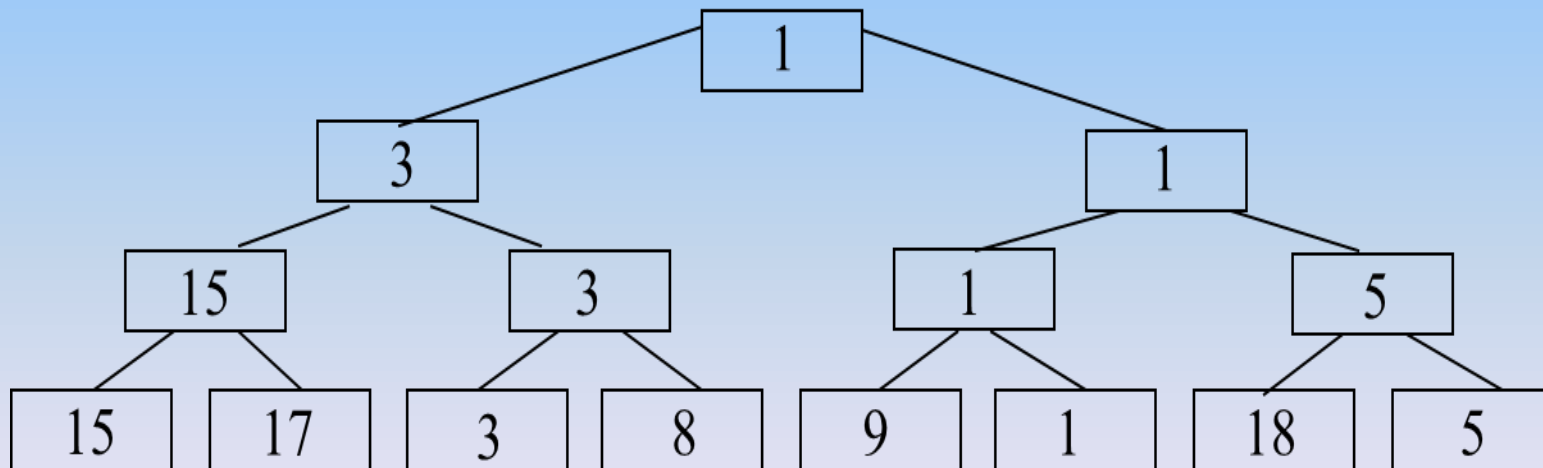


Сортиране. Сортиране чрез избор.

Сортиране чрез избор от дърво:

Изложеният алгоритъм построява **дървото на турнира**, т. е. **определя минималния елемент на масива с точно $n-1$ сравнения:**

$n=8$ $a=\{15, 17, 3, 8, 9, 1, 18, 5\}$





Сортиране. Сортиране чрез избор.

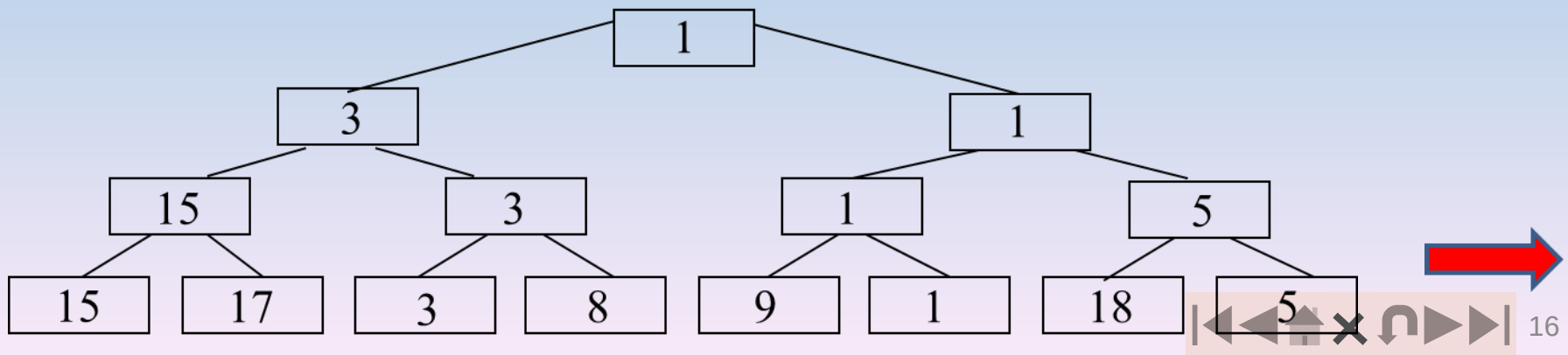
Сортиране чрез избор от дърво:

Сега, заменяйки най-малкия елемент (“шампиона”) с $-\infty$ в съответното му листо, и преизчислявайки съдържанието на върховете по пътя до корена, във върха на дървото ще получим следващия по големина елемент.

Т. к. дървото има височина $\lceil \log_2 n \rceil$, то този процес на определяне на втория по големина елемент (“вицешампиона”) изисква

$\lceil \log_2 n \rceil - 1$ сравнения вместо $n - 2$, (като алгоритъма на пряката селекция).

Така процеса на цялостно сортиране на масива изисква не повече от $n - 1 + (n + 1) * (\lceil \log_2 n \rceil - 1)$ сравнения:





Сортиране. Сортиране чрез избор.

Сортиране чрез избор от дърво:

Изложеният алгоритъм е сериозно подобрение на алгоритъма на пряката селекция, но и той има редица **недостатъци**:

- все още **голям брой на извършваните размени**;
- стойностите $-\infty$, които водят до **излишни сравнения**, като накрая запълват цялото дърво;
- изгражданото дърво, макар и **идеално балансирано**, **не дава оптимални резултати**, т. к. **позволява листата да се появяват на всички нива**: участниците в турнира са разположени в листата, а останалите върхове на дървото само ги **дублират**.

Необходимо е **да се минимизира размера на необходимата памет**, която при алгоритъма на турнира с елиминиране е от порядъка на **$2n-1$** . За целта се търси ефективен механизъм за **линеаризация на дървото**, който **позволява да се пазят само листата, без да се губи известните съотношения между тях**.



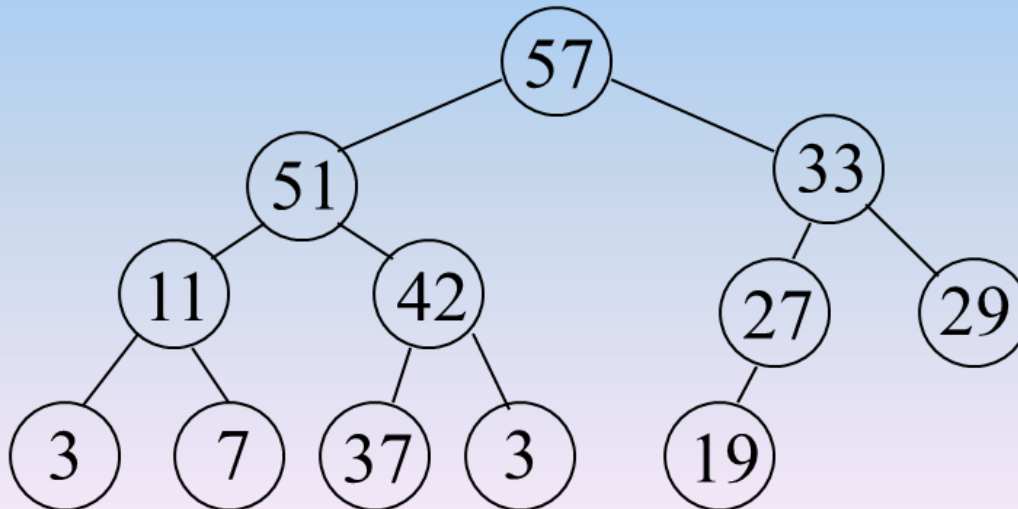
Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

Такъв механизъм е предложен от Дж. Уилямс и се нарича **пирамида**.

Пирамидата е балансирано двоично дърво с височина h , едновременно отговаряща на следните условия:

- 1) всички листа са на ниво h и $h-1$;
- 2) всички **наследници** на даден връх са **по-малки** от него;
- 3) всички **наследници** от ниво h са максимално изместени вляво.





Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

Пирамидата може да бъде ефективно реализирана не само чрез дърво, но и чрез **n -елементен масив**, в който наследствените връзки между елементите се определят като проста **линейна функция на позициите им**.

Как точно?

6 5 3 1 8 7 2 4





Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

Пирамидата е **еквивалентна** на редица от стойности (или ключове):

$a_{lf}, a_{lf+1}, \dots, a_{rt}$ ($1 \leq lf \leq rt \leq n$) $a_i \geq a_{2i}$ и $a_i \geq a_{2i+1}$, $i = lf, lf+1, \dots, rt/2$.

Т.е. представянето е **аналогично** на **статичното представяне на двоично дърво**:

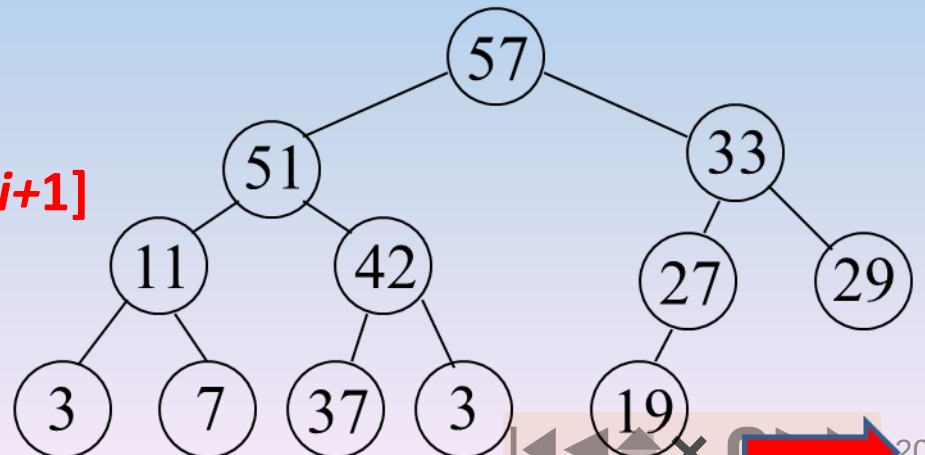
$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	$a[6]$	$a[7]$	$a[8]$	$a[9]$	$a[10]$	$a[11]$	$a[12]$
57	51	33	11	42	27	29	3	7	37	3	19

Левият наследник на $a[i]$ е $a[2i]$

Десният наследник на $a[i]$ е $a[2i+1]$

Върхът на пирамидата

съдържа най-големия елемент.

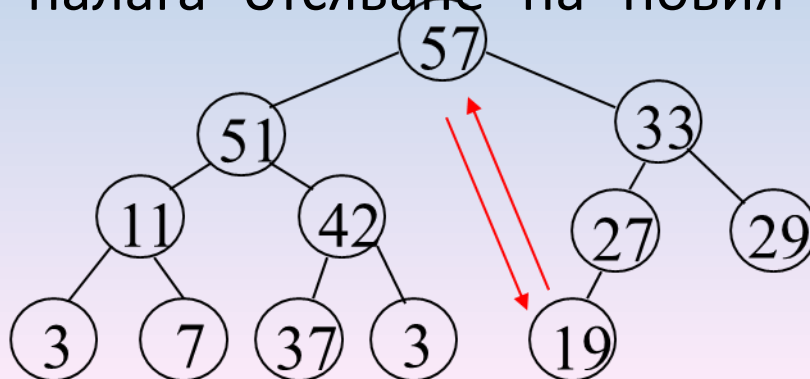




Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

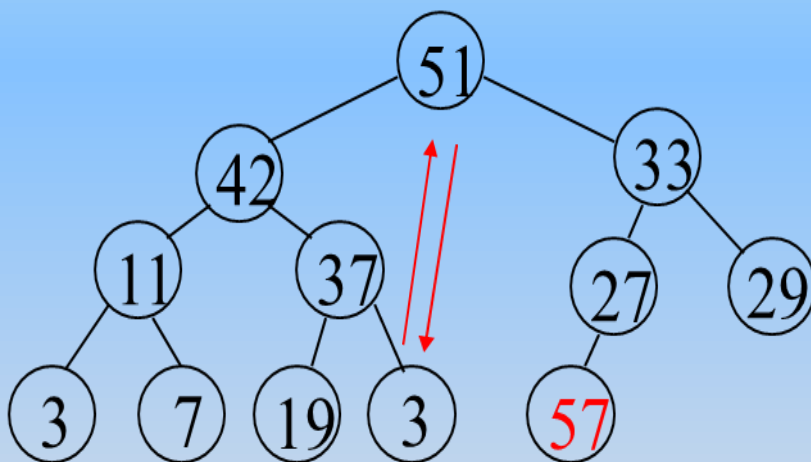
Ако предположим, че разполагаме с ефективен алгоритъм за изграждане на пирамида, реализиран с *build_heap()*, и с алгоритъм за възстановяване на пирамида от *k* елемента *restore_heap()*, след като върхът ѝ е бил заменен с произволен елемент. Тогава, след като изградим пирамида с *n* елемента, може да разменим върха ѝ *a[1]* (най-големия ѝ елемент) с последния ѝ елемент *a[n]*, при което стария връх заема окончателната си позиция - става последен в сортирания масив. Т. к. тази операция нарушава формулираните за пирамидата правила, то се налага отсяване на новия елемент надолу по пирамидата:



Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

В резултат се получава нова пирамида **$a_1, a_2, \dots, a_{n-1}$** , съдържаща един елемент по-малко (без червената стойност):



Следва нова размяна на върха на пирамидата с последния **$(n-1)$** елемент, последвана от ново отсяване. Описаният процес се повтаря **$(n-1)$** пъти (няма нужда от n -та стъпка, защото накрая n -ия елемент си е на място).



Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

Алгоритъмът за пирамидално сортиране е:

...

```
build_heap()  
for (i=n; i>=2; i--)  
{  
    rab=a[1]; a[1]=a[i]; a[i]=rab;  
    restore_heap(i-1);  
}
```

...

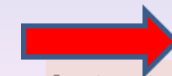
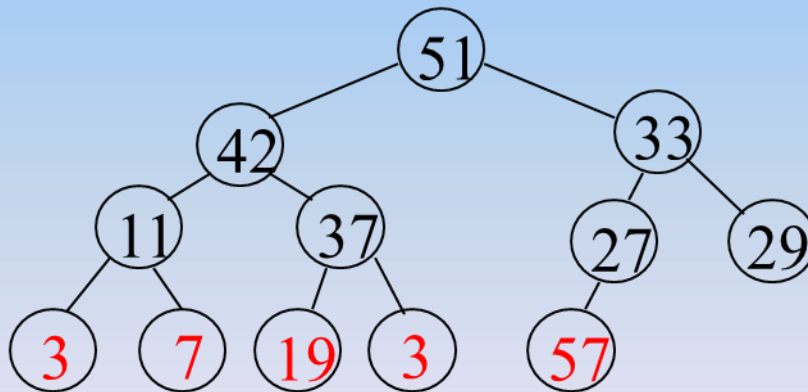


Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

Как да изградим пирамидата?

Нека **n** да е **четно**. Тогава за елементите **$a[n/2+1], a[n/2+2], \dots, a[n]$** не се изисква никакво съотношение за наредбата, т. к. за **НИКОИ** два елемента **i** и **j** не е изпълнено **$j=2i$** и **$j=2i+1$** . Те са листата и за тях не важат зададените правила.



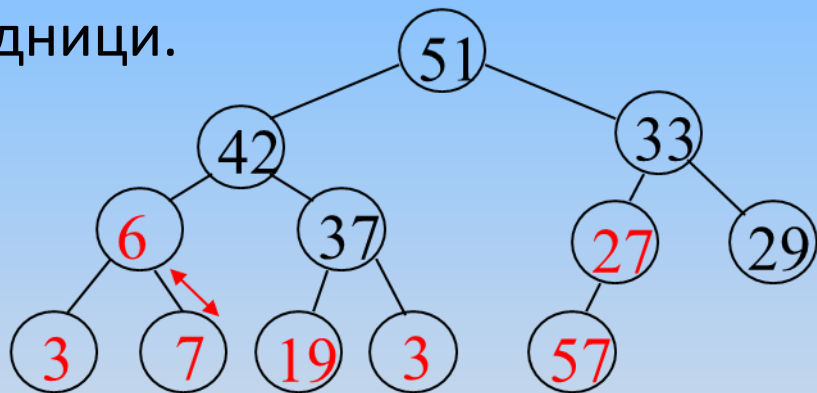


Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

Нека разширим пирамидата отляво с един елемент.

Новодобавеният елемент **x** ще бъде от предпоследното ниво и ще има поне един наследник. Трябва да се проверят съотношенията и при необходимост **x** да се размени с един от своите наследници.



Сега елементите **$a[n/2]$, $a[n/2+1]$, $a[n/2+2]$, ..., $a[n]$** пак образуват пирамида. Следва ново разширяване на пирамидата отляво, и процесът продължава до включване на всички елементи. При нечетно **n** листата са **$a[n/2+1]$, $a[n/2+2]$, ..., $a[n]$** . (В реализацията на C/C++ няма нужда от това разграничение, т. к. операцията $\left\lfloor \frac{n}{2} \right\rfloor$ е целочислена.)

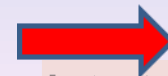


Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

Отсяването на новодобавения елемент **x** надолу по пирамидата може да протича на няколко етапа, т. е. след размяна на **x** с някой от наследниците може да се окаже, че правилата не са изпълнени. Така **x се отсява надолу по пирамидата, като на всяка стъпка потъва с едно ниво надолу, докато не заеме мястото си.**

Тук може да се направи едно очевидно подобрене, ускоряващо процеса, а именно - **x** да се разменя с по-малкия от двата наследника. Освен това размяната на **x** с наследниците му може да се отлага дотогава, докато не бъде намерено точното му място, като вместо това в процеса на търсене на крайната му позиция се извършват еднопосочни присвоявания.



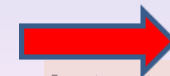


Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

Както построяването, така и възстановяването изискват отсяване на върха надолу по пирамидата. Това означава, че може да се реализира обща функция за отсяване **sift()**:

```
void sift (int a[], unsigned lf, unsigned rt)
{ unsigned i=lf, j=i+i;
  int x=a[i];
  while (j<=rt)
  { if (j<rt)
    if (a[j]<a[j+1]) j++;
    if (x>a[j]) break;
    a[i]=a[j];
    i=j;
    j<<=1;           // умножение по 2
  }
  a[i]=x;
}
```





Сортиране. Сортиране чрез избор.

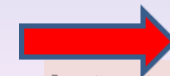
Пирамидално сортиране (Heapsort):

Построяването на пирамидата (**функция `build_heap()`**) може да се извърши така:

```
for (k=n/2+1; k>1; k--)  
    sift(a, k-1, n);
```

За сортирането - многократна размяна на върха на пирамидата с последния ѝ елемент, последвана от отсяване (**функция `restore_heap()`**) получаваме:

```
for (k=n; k>1; k--)  
{ rab=a[1]; a[1]=a[k]; a[k]=rab;  
  sift(a, 1, k-1);  
}
```





Сортиране. Сортиране чрез избор.

Пирамидално сортиране (Heapsort):

Окончателният вариант на пирамидалното сортиране е:

```
void heapsort(int a, unsigned n)
```

```
{
```

```
    unsigned k;
```

```
    for (k=n/2+1; k>1; k--)
```

```
        // построяване на пирамидата
```

```
        sift(a, k-1, n);
```

```
    for (k=n; k>1; k--)
```

```
        //построяване на сортирана  
последователност
```

```
    {
```

```
        rab=a[k];
```

```
        a[k]=a[1];
```

```
        a[1]=rab;
```

```
        sift(a, 1, k-1);
```

```
    }
```

```
}
```



Сортиране. Сортиране чрез сливане.

Сортиране чрез сливане – *Merge sort*:

В основата на тази група алгоритми за сортиране стои вече разгледания метод "**разделяй и владей**": *подрежданата поредица се разделя на две части, всяка от които се сортира, след което подредените подмасиви се обединяват (или се сливат).*

Принципи на "разделяй и владей": разбиване на изходната задача на няколко подзадачи (**разделяй**), решаване на подзадачите и конструиране на решение на изходната задача (**владей**)

Пример: с двоично търсене.

6 5 3 1 8 7 2 4



Сортиране. Сортиране чрез сливане.

Сливане на сортирани масива:

1. Сливане на сортирани масиви:

Как да обединим два подредени по големина масива $a[]$ и $b[]$?

...

```
const an=100, bn=120;
```

```
int a[an], b[bn], c[an+bn];
```

...

Функция, реализираща тази операция:

```
void merge (int a[], int na, int b[], int nb, int c[])
```

```
{ int ia=0, ib=0, ic=0;
```

```
  while (ia<na&&ib<nb)
```

```
    c[ic++]=((a[ia]<b[ib])?a[ia++]:b[ib++]);
```

```
  while (ia<na) c[ic++]=a[ia++];
```

```
  while (ib<nb) c[ic++]=b[ib++];
```

```
}
```

Функцията има един съществен **недостатък**: **не сортира на място**,
а изисква допълнителна памет, колкото за двата сливани масива.



Сортиране. Сортиране чрез сливане.

Сортиране чрез сливане (Mergesort):

2. Сортиране чрез сливане:

Разгледаната функция може да служи за основа на следния прост алгоритъм за сортиране чрез сливане:

*void mergesort (int a[], int n) //функцията разделя масива рекурсивно на две
//части след което функцията merge ги обединява*

```
{ if(n<2) return;  
  int nleft=n/2, nright=n-nleft;  
  mergesort(a, nleft);  
  mergesort(a+nleft, nright);  
  int *p=new int [n];  
  merge (a, nleft, a+nleft, nright, p);  
  for (int i=0; i<n; i++)  
    a[i]=p[i];  
  delete []p; }
```

Между операциите **new** и **delete** няма рекурсивно обръщение и необходимото място в паметта не надминава **$n * \text{size}(T)$** байта (в случая **T** е **int** и необходимия обем памет ще бъде **$4n$**). 



Сортиране. Сортиране чрез сливане.

Сортиране чрез сливане (Mergesort):

Алгоритъмът за сортиране чрез сливане е един от **най-ефективните** от известните алгоритми за сортиране. Гарантираната му **средна времева сложност** е **$O(n \cdot \log_2 n)$** , което теоретически е най-добрата алгоритмична сложност за универсален алгоритъм за сортиране.

Предимства:

- **достъпът до елементите се извършва строго последователно**, т.е. подходящо е за сортиране на списъци, последователни файлове и др.
- сортирането чрез сливане е **устойчиво**, т. е. запазва относителната наредба на елементите с еднакви ключове. (Например, *Quicksort* е неустойчив алгоритъм.)
- алгоритъмът почти **не зависи** от предварителното нареждане на елементите в масива (за разлика, например от *Quicksort*).



Сортиране. Сортиране чрез сливане.

Сортиране чрез сливане (Mergesort):

Основен **недостатък** на сортирането чрез сливане е това, че **изисква допълнителна памет**, пропорционална на n : за допълнителен масив или за указатели.

Подобно повечето съвременни методи за сортиране, сортирането чрез сливане **се държи лошо при малки n** .

Както и в случая на Quicksort, **добри резултати се получават при комбинирането му с други методи**. Стандартен подход е, когато броят на елементите на разглежданото множество падне примерно под 15-20 да се използва сортиране чрез вмъкване. Към групата алгоритми, реализиращи идеята за сортиране чрез сливане, се отнася и **четно-нечетно сортиране чрез сливане на Бетчер**.



Сортиране. Сортиране чрез сливане.

Алгоритъм на Бетчер(четно-нечетно сортиране):

3. Алгоритъм на Бетчер (или Четно-нечетно сортиране чрез сливане по метода на Бетчер - *Batcher' odd-even mergesort*):

Алгоритъмът е разработен през 1968 г. от Бетчер и действа по принципа “разделяй и владей” и е основан само на две абстрактни операции:

- сравнение-размяна;
- перфектно “карторазбъркване” (и обратния ѝ еквивалент – перфектно “картоподреждане”) за преместване на данните.

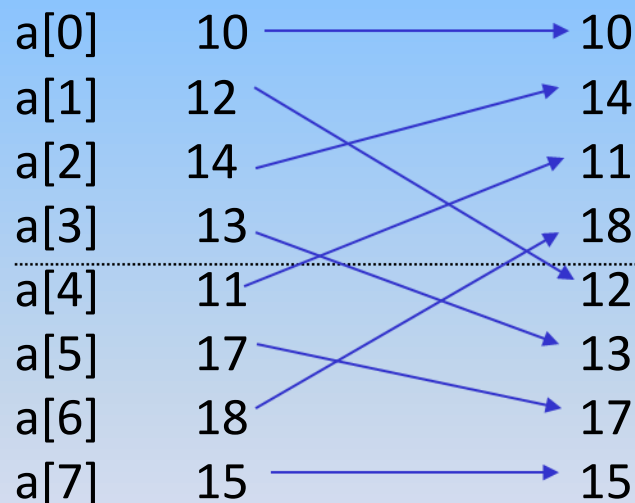
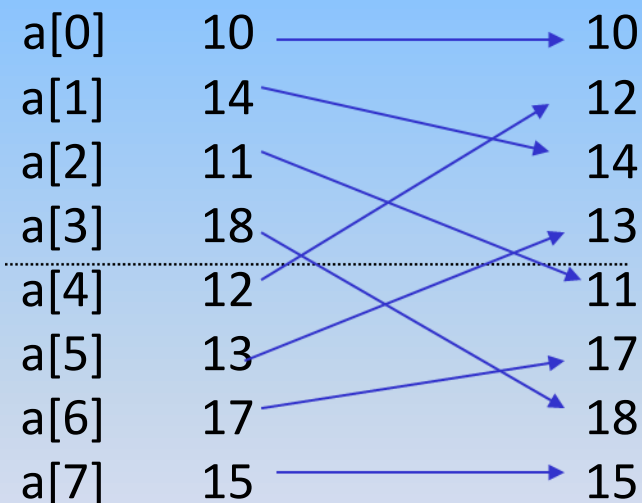


Сортиране. Сортиране чрез сливане.

Алгоритъм на Бетчер(четно-нечетно сортиране):

Първата операция може да бъде реализирана от всеки алгоритъм за сортиране.

Втората операция представлява следното: -перфектно “карторазбъркване”- перфектно “картоподреждане”.



Всички елементи от първата половина на масива ще **имат четни индекси**, от втората - **нечетни**. При обратната операция - елементите с четните индекси отиват в първата половина на масива, с нечетните - във втората.

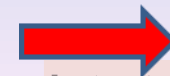


Сортиране. Сортиране чрез сливане.

Алгоритъм на Бетчер(четно-нечетно сортиране):

Перфектното карторазбъркване пренарежда масива по начин, **съответстващ** на начина, по който **тесте карти би могло да се пренареди**, когато се разбърква от професионалист.

То се **разцепва** прецизно **на две половини**, после се прави “розетка”, така че картите от двете половини се вземат алтернативно, за да **направят разбърканото тесте**. **Първата карта винаги се взема от горната половина**. Ако **броят** на картите е **четен**, двете половини имат **еднакъв** брой карти; ако **броят** на картите е **нечетен**, допълнителната карта е в **края на горната половина**.



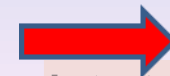


Сортиране. Сортиране чрез сливане.

Алгоритъм на Бетчер(четно-нечетно сортиране):

Функция **shuffle**, извършваща перфектно карторазбъркване:

```
void shuffle (int a[], int lf, int rt)
{
    int i, j, m=(lf+rt)/2;
    static aux[maxn];           // временен масив
    for (i=lf, j=0; i <=rt; i+=2, j++)
    {
        aux[i]=aux[lf+j];
        aux[i+1]=a[m+1+j];
    }
    for (i=lf; i<=rt; i++)
        a[i]=aux[i];
}
```





Сортиране. Сортиране чрез сливане.

Алгоритъм на Бетчер(четно-нечетно сортиране):

Функция ***unshuffle***, извършваща перфектно картоподреждане:

```
void unshuffle (int a[], int lf, int rt)
```

```
{
```

```
    int i, j, m=(lf+rt)/2;
```

```
    static aux[maxn];
```

// временен масив

```
    for (i=lf, j=0; i <=rt; i+=2, j++)
```

```
    {
```

```
        aux[lf+j]=aux[i];
```

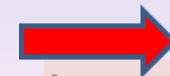
```
        a[m+1+j] = aux[i+1];
```

```
    }
```

```
    for (i=lf; i<=rt; i++)
```

```
        a[i]=aux[i];
```

```
}
```





Сортиране. Сортиране чрез сливане.

Алгоритъм на Бетчер(четно-нечетно сортиране):

Нечетно-четно сливане на Бетчер:

```
merge_B(int a[], int lf, int rt)
{
    int i, m=(lf+rt)/2, rab;
    if (rt==lf+1)
    {
        rab=a[lf];
        a[lf]=a[rt];
        a[rt]=rab;
    }
    if (rt<lf+2) return;
    unshuffle(a, lf, rt);
    merge_B(a, lf, m);
    merge_B(a, m+1, rt);
    shuffle(a, lf, rt);
    for (i=lf+1; i<rt; i+=2)
    {
        rab=a[i]; a[i]=a[i+1]; a[i+1]=rab;
    }
}
```



Сортиране. Сортиране чрез сливане.

Алгоритъм на Бетчер(четно-нечетно сортиране):

Самото сливане на Бетчер е рекурсивен метод “**разделяй и владей**”.

За да направим **сливане 1-към-1**, просто използваме операция “сравнение-размяна”. От друга страна, за да направим **n -към- n сливане**, ние извършваме **картоподреждане**, за да получим **две $n/2$ -към- $n/2$ задачи** за сливане, после ги решаваме рекурсивно, за да получим два сортирани масива. Като извършим карторазбъркване на тези два масива, получаваме масив, който е почти сортиран - всичко, което е необходимо да направим, е да **преминем още веднъж през масива с $n/2 - 1$ на брой независими операции “сравнение-размяна”**: между елементи **$2i$ и $2i+1$** за i от 1 до **$n/2 - 1$** .



Сортиране. Сортиране чрез разпределение.

Сортиране чрез разпределение:

Методът, за разлика от разгледаните досега, **не изисква сравнение на целите ключове**.

Предполага се, че множеството на ключовете не е голямо и всеки ключ има следния вид:

$a_1 a_2 \dots a_{k-1} a_k$

където всеки компонент a_i е елемент на някакво крайно множество (например - латинската азбука).

Компонентите се **сравняват отдясно наляво** в лексикографски ред:

```
s d a f g z < a s b f g z
a s b f g z
```

Самото сортиране на ключовете се извършва с помощта на друг устойчив алгоритъм за сортиране.



Сортиране. Паралелно сортиране.

Разгледаните до тук алгоритми за сортиране не позволяват извършването на повече от една операция (сравнение) едновременно.

Усъвършенстването на технологиите за производство, понижаването на разходите и свързаното с него понижение на цените на процесорите направи възможно все по-масово използването на **многопроцесорните системи и клъстери**. Масовото им навлизане, съчетано с нарастването на броя на процесорите, води до повишен интерес към алгоритмите за паралелни изчисления. Налице е вътрешно хеширане и изпълняване на повече от една операции едновременно, като изпълнението на всяка **следваща операция започва веднага**, след като се освободят съответните ресурси в процесора, **без да се чака окончателното завършване на предходната**.



Сортиране. Паралелно сортиране.

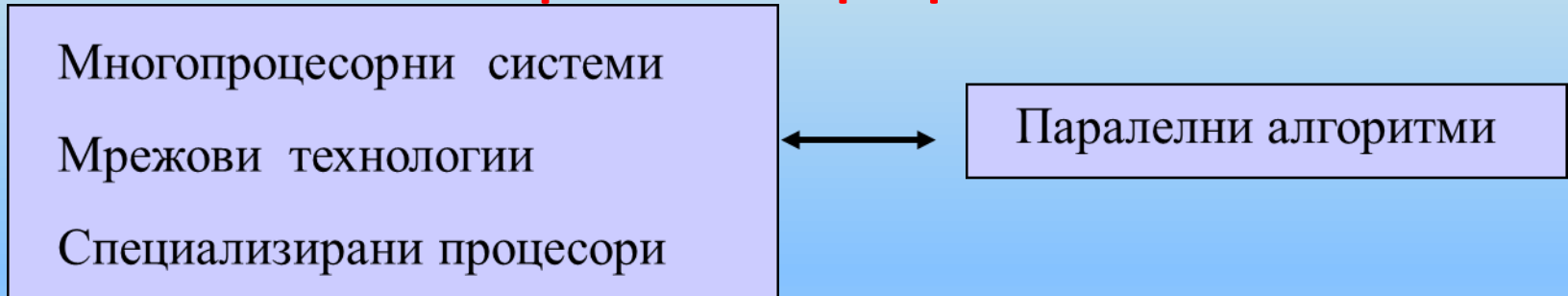
За пълното разгръщане възможностите на паралелния хардуер са нужни софтуерни средства за управление процеса на декомпозиция (разбиване) на задачата и разпределение на отделните подзадачи между паралелните хардуерни устройства (най-често процесори) за тяхната обработка. Т. е. възниква нужда от паралелни алгоритми.

Съществуват редица алгоритми, които лесно се паралелизират, докато други се поддават трудно или пък изобщо не се поддават. Нещо повече: **универсален алгоритъм за паралелизиране не съществува**. Въпреки че съществуват някои общовалидни схеми, които улесняват процеса, по принцип във всеки конкретен случай следва да се подхожда по различен начин.



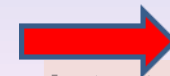
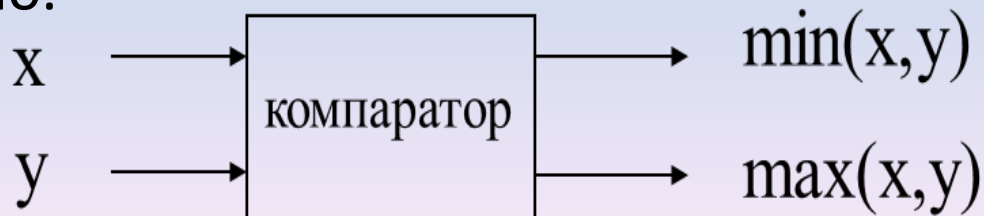
Сортиране. Паралелно сортиране.

Паралелно сортиране:



Абстрактна машина, представляваща модул за сравнение-размяна или компаратор:

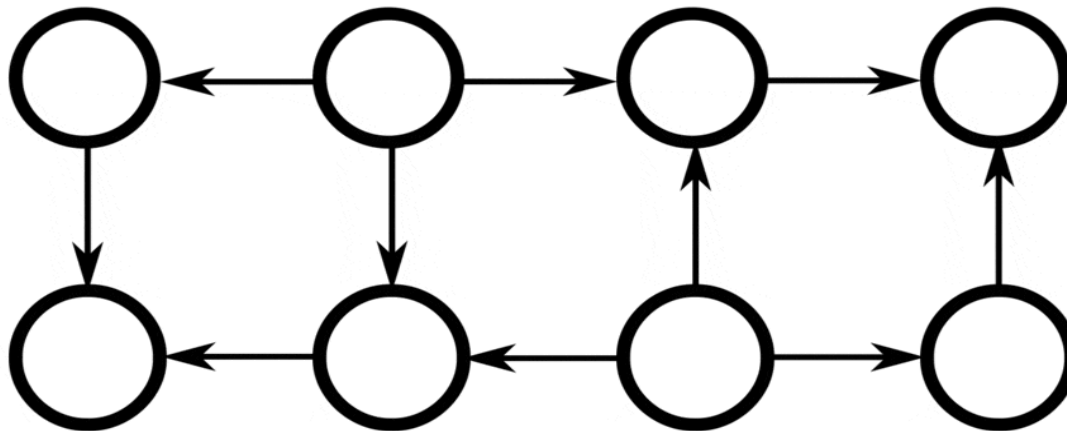
Когато на входа на компаратор постъпят две числа x и y , на първия изход излиза **$\min(x,y)$** , а на втория — **$\max(x,y)$** . Ще считаме, че разполагаме с достатъчен брой компаратори, както и, че произволен брой от тях могат да работят едновременно, т. е. паралелно.





Сортиране. Паралелно сортиране.

Паралелно сортиране:

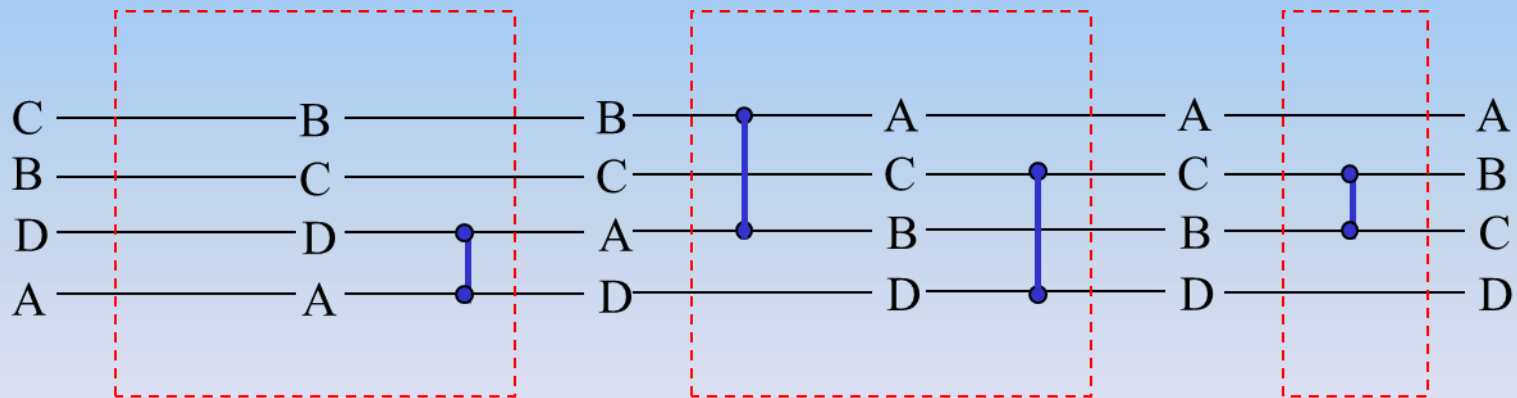
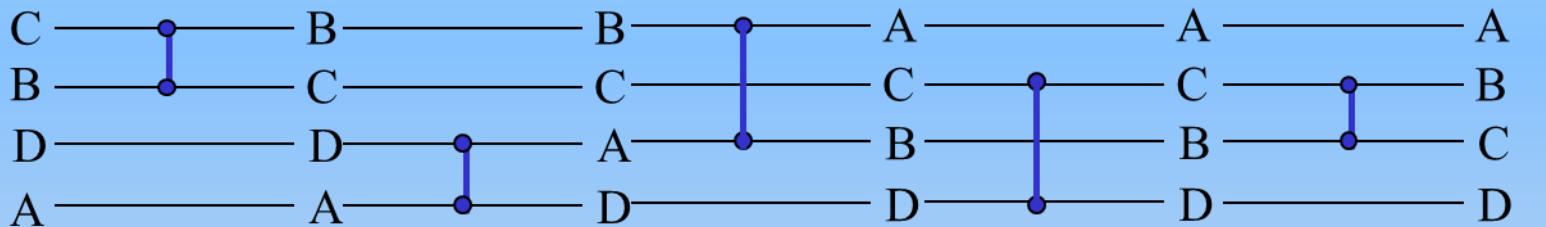




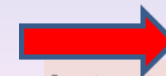
Сортиране. Паралелно сортиране.

Паралелно сортиране:

Компараторите (броят им не се ограничава) се свързват в сортираща схема или мрежа:



паралелни фази

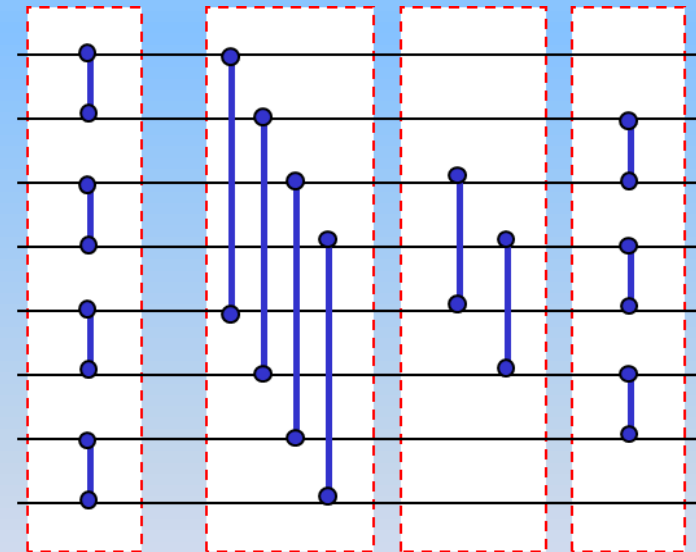
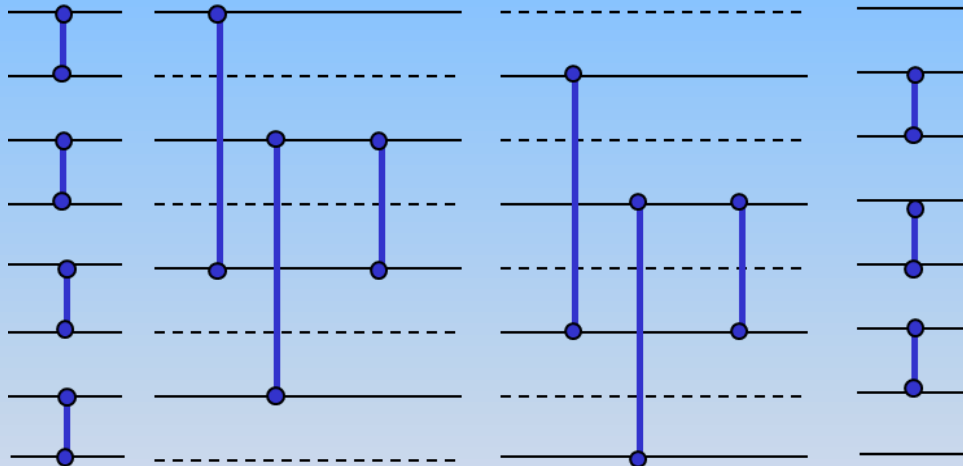




Сортиране. Паралелно сортиране.

Паралелно сортиране:

Сливаща мрежа за сортиране на 8 елемента:

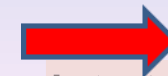
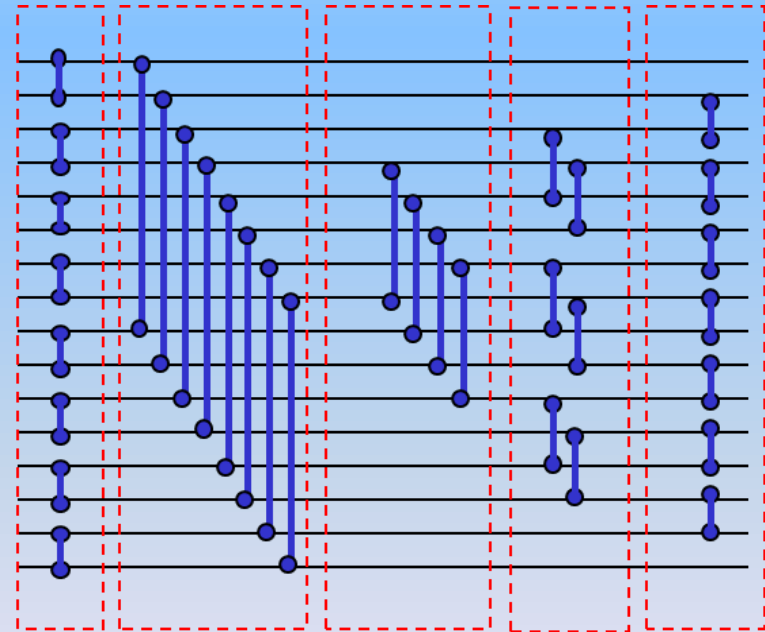
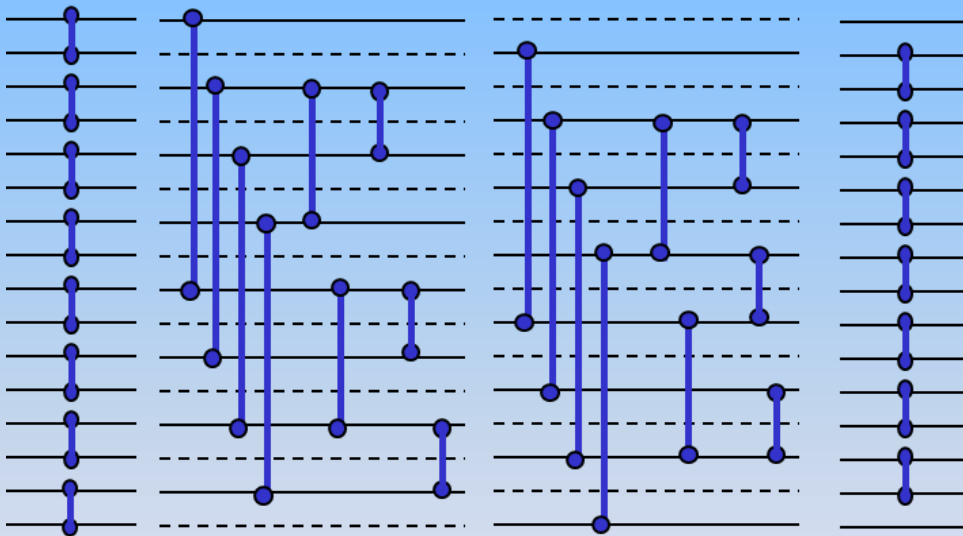




Сортиране. Паралелно сортиране.

Паралелно сортиране:

Сливаща мрежа за сортиране на **16** елемента:

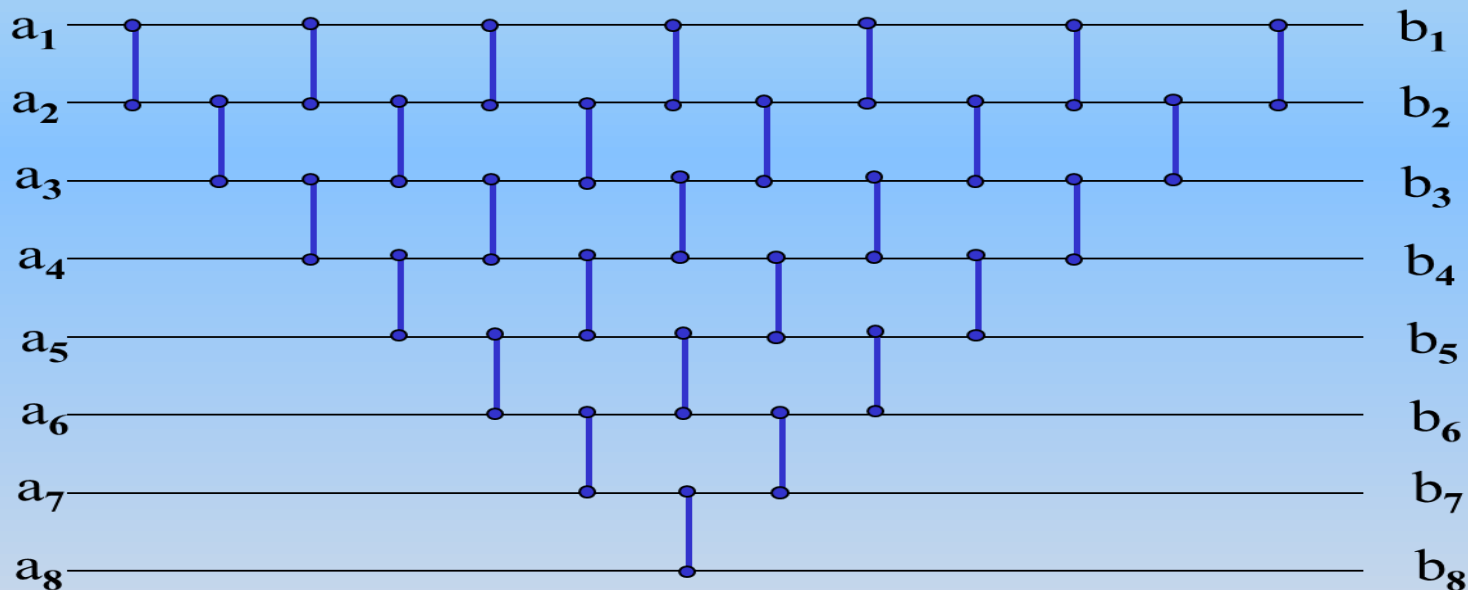




Сортиране. Паралелно сортиране.

Паралелно сортиране:

Сортираща схема за сортирането чрез вмъкване:



Тъй като при сортирането чрез вмъкване се извършва сравнение само между съседни елементи, компараторите от горната схема свързват само съседни линии. Подобна е ситуацията и при метода на мехурчето.



Сортиране. Външно сортиране.

Външно сортиране:

Разгледаните до тук методи за сортиране са предназначени за **подреждане** на масиви от данни, **намиращи се в ОП**. Но бихме могли да подреждаме и файлове, **намиращи се на някакъв външен носител**. За тази цел са необходими **алгоритми за външно сортиране**.

Разликата между методите за вътрешно и външно сортиране е по-скоро **прагматична, отколкото фундаментална**. Т. к. С/С++ позволява работа с файлове в режим на произволен достъп, бихме могли да **адаптираме вече разгледаните методи и за файлове**.



Сортиране. Външно сортиране.

Външно сортиране:

Но трябва да се имат предвид няколко момента:

- Повечето методи за вътрешно сортиране сортират *in situ* (на място), т. е. **използват само един масив**. Както данните, които трябва да бъдат сортирани, така и подредените вече данни са в този масив и **не са нужни временни масиви**. От друга страна, **външните методи обикновено не са толкова икономични по отношение на паметта, а използват няколко файла**.
- Последователния достъп до елементите на масив обикновено **не е по-бърз** от произволния достъп до масив. **Това не е така при файловете**, т. к. при последователния достъп до файлове големи количества данни могат да бъдат **съхранявани в буфер**. Така че **използването на последователен достъп при външно сортиране може да се окаже по-бързо**.



Сортиране. Външно сортиране.

Външно сортиране:

Балансирано многоходово сливане или BMM (Balanced multiway merging):

6 временни файла или ленти: **3 за вход и 3 за изход**

Входният файл съдържа следните данни:

ereisyourbooktheoneyourthousandsoflettershaveaskedmetopublish

N=5

**herei syour bookt heone yourt housa ndsof lette rshav easke
dmeto publi sh**

Лента 0: **eehir eehno dfnos aeeks hs**

Лента 1: **orsuy ortuy eeltt demot**

Лента 2: **bkoot ahosu ahrrsv bilpu**





Сортиране. Външно сортиране.

Външно сортиране:

Лента 0: **eehir eehno dfnos aeeks hs**

Лента 1: **orsuy ortuy eeltt demot**

Лента 2: **bkoot ahosu ahsv bilpu**

$a[0]=e$ $a[1]=o$ $a[2]=b$ $a[2]=k$

Лента 3: **beehikoorrstuy abdeeeiklmopstu**

Лента 4: **aeehhnoorstuuy hs**

Лента 5: **adeefhlnorssttv**

Лента 0: **aabdeeeeeefh h h h i k l n n o o o o o o o r r r r s s s t t t t u u u v v y y**

Лента 1: **abdeeeh i k l m o p s s t u**

Лента 2:

Окончателен резултат (Лента 3):

aaabbddeeeeeeeeeefh h h h h i i k k l m n n o o o o o o o o p r r r r s s s s s t t t t t u u u u v v y y